Computers & Geosciences Vol. 18, No. 8, pp. 989–1001, 1992
Printed in Great Britain. All rights reserved

0098-3004/92 $5.00 + 0.00
Copyright © 1992 Pergamon Press Ltd

AN ALGORITHM FOR CALCULATING MINIMUM EUCLIDEAN DISTANCE BETWEEN TWO GEOGRAPHIC FEATURES

DONNA J. PEUQUET
Department of Geography and Earth Systems Science Center, The Pennsylvania State University, University Park, PA 16802, U.S.A.

Abstract—An efficient algorithm is presented for determining the shortest Euclidean distance between two features of arbitrary shape that are represented in quadtree form. These features may be disjoint point sets, lines, or polygons. It is assumed that the features do not overlap. Features also may be intertwined and polygons may be complex (i.e. have holes).

Utilizing a spatial divide-and-conquer approach inherent in the quadtree data model, the basic rationale is to narrow-in on portions of each feature quickly that are on a facing edge relative to the other feature, and to minimize the number of point-to-point Euclidean distance calculations that must be performed. Besides offering an efficient, grid-based alternative solution, another unique and useful aspect of the current algorithm is that is can be used for rapidly calculating distance approximations at coarser levels of resolution.

The overall process can be viewed as a top-down parallel search. Using one list of leafcode addresses for each of the two features as input, the algorithm is implemented by successively dividing these lists into four sublists for each descendant quadrant. The algorithm consists of two primary phases. The first determines facing adjacent quadrant pairs where part or all of the two features are separated between the two quadrants, respectively. The second phase then determines the closest pixel-level subquadrant pairs within each facing quadrant pair at the lowest level. The key element of the second phase is a quick estimate distance heuristic for further elimination of locations that are not as near as neighboring locations.

Key Words: Distance, Proximity problems, Quadtrees, Geographic Information Systems.

INTRODUCTION

Automated Geographic Information Systems (GIS) that store, display, and manipulate geographic data have become an essential tool for a wide range of governmental and private organizations. One fundamental task in a GIS context is the calculation of the minimum distance between two features. Example applications include determining the distance between a newly discovered toxic waste site and a nearby residential area in order to determine potential impact, or to evaluate the change in distance between a growing urban/residential area and an ecologically sensitive wildlife habitat through successive temporal observations. Minimum distance between two features also has importance in a range of other application contexts, such as in Robotics where the calculation of minimum distance is needed for collision avoidance as a robot moves around or manipulates specific static objects and in medical image processing for evaluating changes in bone mass in joints. Identifying the specific point location on each feature that is closest with respect to the other feature also is an essential component of this task. Indeed, this information may be just as important as the actual distance measure for some applications.

Performing this task within a GIS usually is achieved at present by a region expansion operation, radially spreading outward from one feature uniformly and counting constant increments until the first location occupied by the other object is encountered (Tomlin, 1990). Each distance increment is counted using manhattan, chessboard, or other distance metric as an approximation of Euclidean distance. This process and the use of the associated metrics was described originally by Rosenfeld and Pfaltz (1968). They calculated a time complexity of $O(dc)$, where d = the number of increments and c = the grid connectivity associated with the distance metric used. For example, manhattan distance has 4-connectivity and chessboard distance has 8-connectivity. This was improved recently upon with the use of quadtrees by Ang and Samet (1988) and Ang, Samet, and Shaffer (1990). This algorithm utilizes the hierarchical nature of quadtrees and exhibits an execution time that generally decreases as the radius of expansion increases.

The expansion approach is the one most widely used in a GIS context for determining distance as well as for locating nearest neighbor, but this approach is best suited for standard buffering operations where all features within a prespecified distance in all directions are to be retrieved (Tomlin, 1990). Although

there is appeal in the parsimony of using a multipurpose approach for handling all or most proximity problems, it is intrinsically an inefficient solution for determining minimum distance between two given features. The inefficiency arises for calculating minimum distance because the operation expands one feature in all directions, regardless of the relative positions of the two features. Thus if minimum distance between features is a function frequently performed or if the features are large and complex, a more tailored procedure is needed.

Specific methods for determining the minimum distance between two features is an area that has been widely studied within the fields of Mathematics and Computational Geometry. Existing solutions utilize either a chamfering method or employ the construction of a Voronoi diagram. The basic idea of the chamfering method is to limit the measurement area between the two features at any given step to a wedge-shaped area radiating out from a point on one toward the other feature (Forsgren and Seideman, 1990; Bobrow, 1989). This has two disadvantages: first, it requires a priori information regarding their relative positions (e.g. x–y centroid locations), second, it is mathematically complex.

The most widely known approach for solving the minimum distance problem is to construct a Voronoi diagram and then identify the segments whose Voronoi sources are from the two different subsets (i.e. feature A vs feature B), and then determine the minimum Euclidean distance between these two subsets (Wang and Chan, 1986). Both the divide-and-conquer and plane sweep approaches for constructing the Voronoi diagram, described by Widmayer, Wu, and Wong (1985) and Fortune (1986), respectively, have elements that are complex, although the basic ideas are straightforward. Because of this, most implementations for computing the Voronoi diagram rely on a less efficient incremental approach (Guibas and Stolfi, 1985).

Both chamfering and Voronoi construction solutions for calculating distance between two features also are vector-oriented in that the input data are assumed to be two sets of x–y point coordinates that represent the boundary vertices of each of the two features. Identifying facing portions of the two boundaries and dealing with potential nonconvexity in this context are major determinants of algorithm efficiency and conceptual complexity.

The algorithm presented in this paper provides an alternative solution for features represented in a hierarchical grid (i.e. quadtree) form. This algorithm is straightforward conceptually and efficient. It correctly determines the minimum Euclidean distance between any two features of arbitrary shape and relative position directly without the need to construct an explicit proximity structure or need for a priori information about relative orientation before being able to determine closest point pairs.

The use of a recursive spatial subdivision approach for the current algorithm allows the quick elimination of cells that are interior to a polygon and to quickly narrow-in on portions of each feature that are on a facing edge relative to the other feature in order to determine the two closest points. This represents a significant improvement in calculating minimum distance to previous grid-based expansion approaches. A specific advantage of the current algorithm, particularly for large geographic database applications, is that it has the capability to calculate quick distance approximations at some selected coarser level of resolution.

In order to provide some insights into the development of the current algorithm, the basis for the approach used is discussed here. This is followed by a brief overview of the spatial addressing scheme employed before the description of the algorithm itself is given.

THE CLOSEST PAIR PROBLEM

It is readily apparent that minimum distance between features is a variant of the closest point pair problem, as are most proximity problems. The basic task in the situation of two polygons, for example, is to determine the locations (i.e. point) on each polygon, among all locations, that is closest to the other polygon. The closest pair problem usually is defined as follows: Given a set of points P in k-dimensional space, locate the two points within P where the distance between them is smallest. The closest pair problem is a classical problem in Computational Geometry because a number of other problems, including nearest neighbor, incorporate it as part of their solution (Preparata and Shamos, 1985; Bentley, 1980).

An $O(n \log n)$ divide-and-conquer algorithm for solving the closest pair problem was described first by Shamos (1978). This algorithm generally can be described for the two-dimensional case as follows: given a set of points, P, partition P into two subsets, P_1 and P_2, along a vertical line defined by the median vertical coordinate. This step is shown graphically in Figure 1, where P_1 and P_2 are separated by the solid vertical line. If there are more than two points in either of the resultant subsets, subdivide those subsets and recursively apply this procedure to those subsets. Otherwise, determine the Euclidean distance in each subset. The closest pair of points between two subsets always will be:

$$d = \min(|p_{1a} - p_{1b}|, |p_{2a} - p_{2b}|).$$

The closest pair in the whole set is either $\{p_{1a}, p_{1b}\}$, $\{p_{2a}, p_{2b}\}$, or some $\{p_{1c}, p_{2c}\}$, where $p_{1c} \in P_1$ and $p_{2c} \in P_2$. For a more detailed discussion of this algorithm as well as various proximity problems, the reader is referred to Preparata and Shamos (1985).

There is a critical difference between the closest pair problem as just defined and determining the two

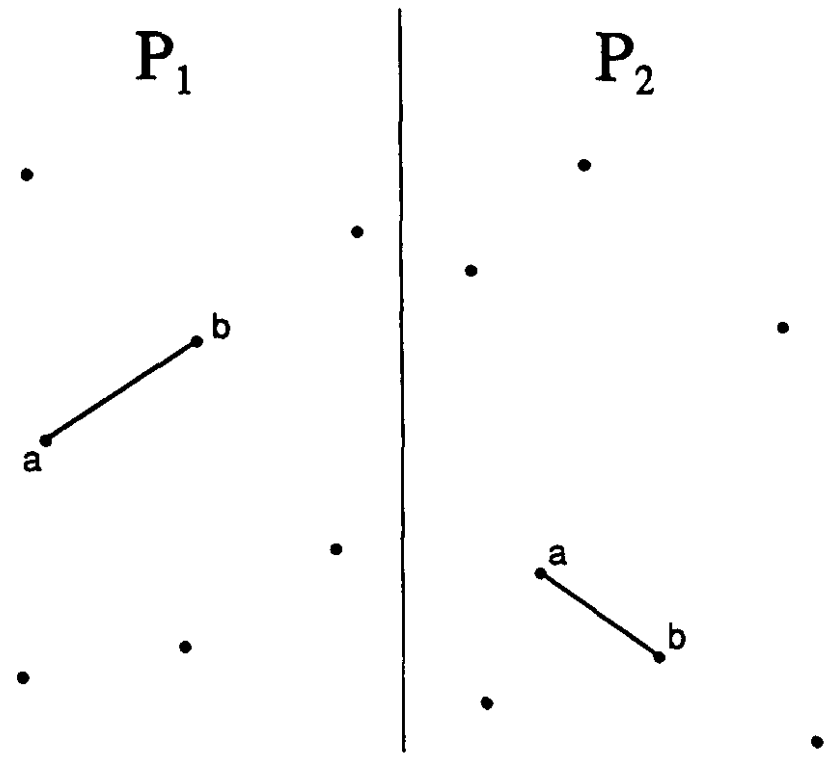


Figure 1. Divide and conquer strategy for locating closest pair of points within set of points (after Shamos, 1978).

closest locations between *features*, where these features can be polygonal, linear, or point sets. The latter problem involves not locating the closest two points within a single set of points, but rather locating the two closest points between two distinct sets of points, because linear and polygonal features also can be viewed as point sets in a gridded structure. This latter observation can be seen in Figure 2 where a line and a polygon are each represented as a line drawing in Figure 2A. In gridded form, the areal extent of these features are discretized into sets of individual cells, as shown in Figure 2B. For the purpose of the present discussion, it is assumed that grid cells are sufficiently small that individual cells can be considered equivalent to point locations.

The minimum Euclidean distance between two features, A and B, such as in the example of Figure 2, is defined as; $d(A, B) = \min\{d(a, b) | a \in A, b \in B\}$. The minimum distance between point a and B is defined as; $d(a, b) | b \in B\}$. Similarly, the minimum distance between b and A is defined as; $d(a, B) = \min\{d(b, A) = \min\{d(a, b) | a \in A\}$. It also is assumed that the distance relation has the following properties; if $d(a, b) = d(b, a)$, then $d(A, B) = d(B, A)$ (Symmetry). Also, $d(A, B) \geqslant 0$ and $d(A, B) = 0$ **iff** $a = b$ (Positive definiteness). This latter property indicates that features do not overlap.

Noting that an edge in the current context can be either a polygon boundary, a line, or a bounding vector around a point set, it is seen that a must be somewhere in the portion of the edge of A that faces B, and b must be somewhere on the portion of the edge of B that faces A. It is this latter observation that has led to a number of solutions to the closest pair between point sets problem using only the edges represented in vector form as ordered sequences of coordinates. Schwartz (1981) gives such an algorithm with $O(\log^2 n)$ time complexity for determining the closest pair of points between two convex closed edges by a double binary search on the coordinate sets. This time was improved later to $O(\log n)$ (Chin and Wang, 1984).

The algorithm described in the current paper is an efficient solution that is not limited to convex shapes. Similar to the Shamos closest point pair algorithm, a recursive spatial subdivision strategy is used to eliminate quickly portions of the two features that have no possibility of containing candidate minimum distance points. The current algorithm relies on the hierarchical nature of quadtrees and additional characteristics of the leafcode addressing structure to achieve this.

In the following section, the quadtree structure and leafcode addressing scheme are briefly reviewed. In the remainder of this paper, the new distance algorithm is described and then presented in pseudocode. Following this, an analysis of the algorithm is given.

THE LEAFCODE ADDRESSING SCHEME

The area quadtree, first described as a spatial data model by Klinger (1971), recursively subdivides a

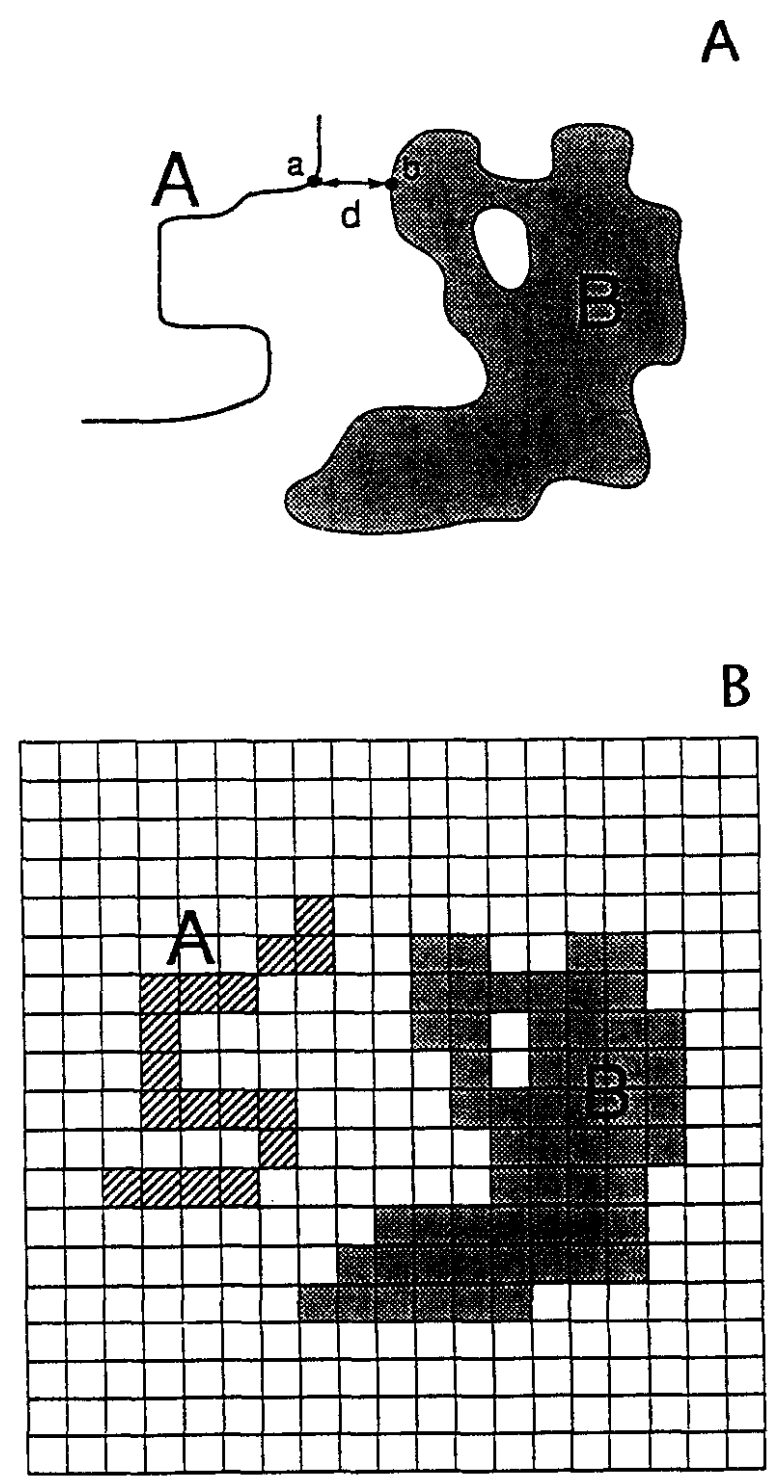


Figure 2. Minimum Euclidean distance between line and polygon represented in both A, line drawing and B, gridded form.

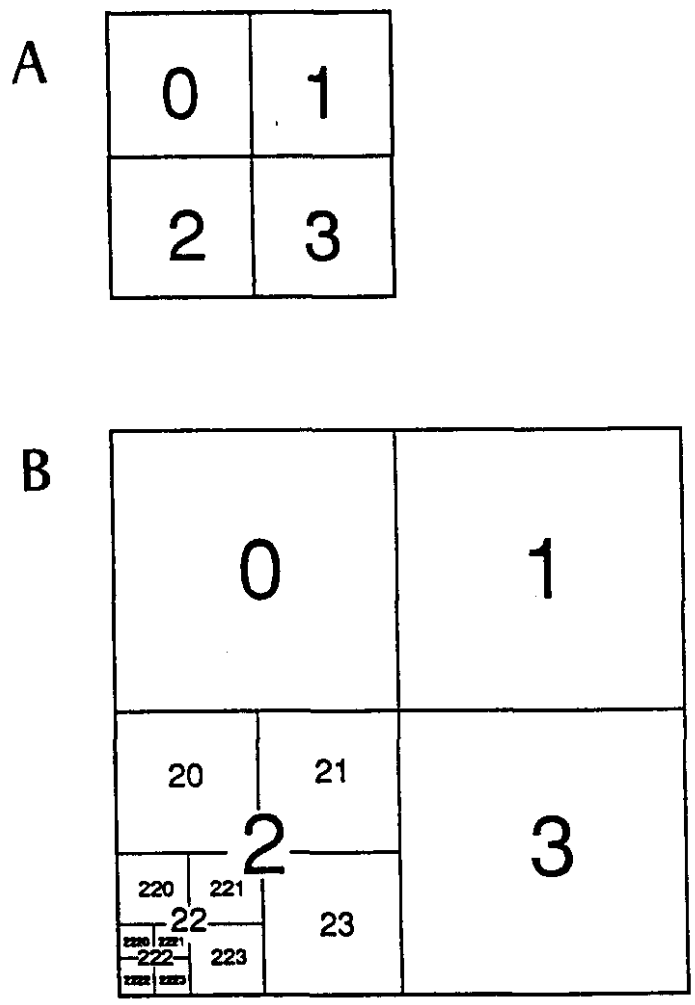


Figure 3. Leafcode addressing structure. A—Quaternary leafcode digits; B—hierarchical scheme.

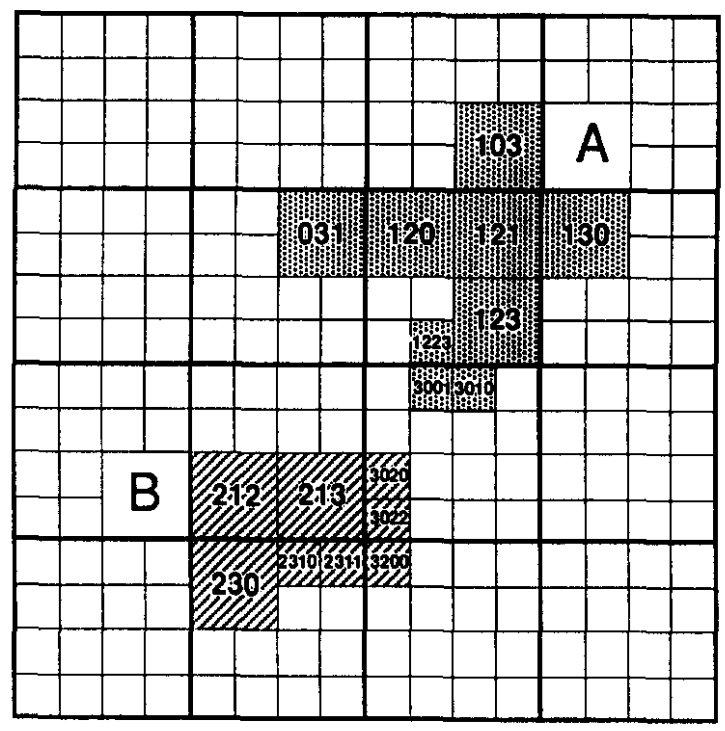
square area into four smaller squares of equal area. This translates functionally into a tree of out-degree 4. The quadtree structure has been studied subsequently and algorithms developed by many others particularly for use in GIS. For a comprehensive survey, the reader is referred to Samet (1989a, 1989b). Quadtrees are proving not only to be efficient for the storage and retrieval of geographic data, they also are versatile in handling a wide range of analytical tasks.

Quadtrees can be implemented either by utilizing an explicit pointer structure or a locational addressing scheme. The latter have become known as linear quadtrees because individual geographic features can be represented simply as lists of the addresses of all component cell locations. This avoids the space overhead encountered with any pointer mechanism. Gargantini described an addressing scheme that is based on a specific Peano scan known as the Morton Matrix (Morton, 1966; Gargantini, 1982). Specific quadtree addresses utilizing this scheme are known as leafcodes.

The leafcode addressing structure as used here consists of a sequence of quaternary digits, where each digit corresponds to the NW, NE, SW, and SE quadrants as shown in Figure 3A. With each successive subdivision in the quadtree, the number of the quadrant is added to the right of the address for the parent quadrant. Each digit in the address thus corresponds to a specific quadrant at a specific level in the quadtree as shown in Figure 3B. For any leafcode address, the most significant digit refers to the quadrant of the first subdivision (i.e. level 1 of the quadtree), and for n subdivisions, n digits will be needed to indicate the location of a node at the nth level. The top level, bounding square is the root (level 0) and does not require a digit in the current context. A terminating character also is needed to indicate the end of the address in the right side of any actual implementation. The most convenient character to use differs with the computer language and the application context. As an example, NUL was used for the C language implementation of the current algorithm. A graphic example of two figures are shown with their corresponding leafcode lists in Figure 4.

Leafcode addresses also use a mechanism known as maximal block encoding to minimize the total number of addresses needed to define any individual feature. The idea is to record only the address of the highest level block that is covered completely by a given feature. Descendant nodes in this situation are not recorded. This utilizes the hierarchical nature of quadtrees to represent larger homogeneous areas as a single address.

This numbering scheme and related schemes for other regular tessellations have been shown to have useful characteristics for performing adjacency, directional vector, union and intersection as well as other calculations (Gargantini, 1982; Peuquet, 1984, 1988; Samet 1982, 1989a; van Lierop, 1986).



B	A
212	031
213	103
230	120
2310	121
2311	1223
3020	123
3022	130
3200	3001
	3010

Figure 4. In subdividing area containing two features, subquadrant pairs 1–2 will be processed directly by next phase of algorithm to locate closest point pair between those subquadrants. Subquadrant 3 is subdivided once more before closest point pairs are determined.

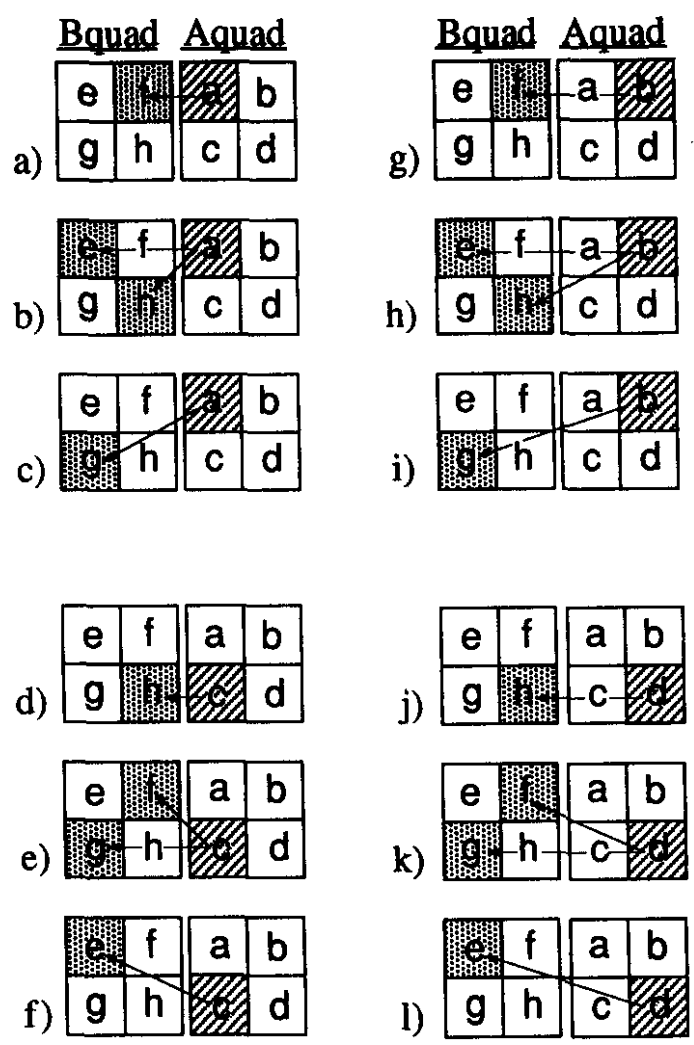


Figure 5. Nearness hierarchy used in selecting "nearest" candidate subquadrant pairs within cardinally adjacent quadrants.

CALCULATING MINIMUM DISTANCE BETWEEN TWO FEATURES

The overall logic

The basic logic for the closest point pair solution can be extended in order to derive a straightforward solution for shortest Euclidean distance between two polygons, lines, points, or any combination of these. The key, as previously stated, is to view the two features as two sets of areal cells, each set representing the complete areal extent of one feature.

The algorithm has two primary phases. The first phase quickly locates pairs of adjacent quadrants where some portion of each feature is somewhere within a separate but adjacent quadrant from the other feature. Adjacent quadrants include diagonal pairs as well as neighbors in the cardinal directions (i.e. is 8-connected). The primary purpose of this phase is to determine the relative orientation and location of facing edges. It also has the effect of possibly eliminating portions of a feature from further consideration if these portions are far enough away from any facing edge that they do not fall within any adjacent quadrant. The second phase locates the paired facing edge locations that are candidate "nearest" edge locations, and reduces them to lowest level leafcode locations. This second phase relies on a "quick estimate" distance measure for further elimination of locations that are not as near as neighboring locations. This serves to minimize the number of actual distance calculations and quantitative comparisons that must be made as a final test for determining the closest two points.

Visualizing a configuration such as the example polygons *A* and *B* shown in Figure 4, the overall conceptual logic of the algorithm is as follows:

(1) Locate adjacent quadrant pairs. Starting at the top level of the quadtree hierarchy, compare relative quadrant locations, among sixteen possible pair combinations of quadrants (e.g. 0 and 0, 0 and 1, 0 and 2, etc.). If any portion of *both* features lie within the *same* subquadrant at the current level, then subdivide this quadrant and recursively repeat step (1) to locate adjacent quadrant pair(s) within it. If, on the other hand, there is some portion of one feature in one quadrant and some portion of the other feature in the other quadrant, we have adjacent occupied quadrants. Addresses of *A* and *B* within this quadrant pair are passed to phase (2). Any pair combination where one or both of the quadrants is empty, that quadrant pair is eliminated from further processing.

Adjacent quadrant pairs can be at any level of the quadtree hierarchy, but both quadrants of any given pair will be at the same level. Thus, in the example shown in Figure 4, only addresses for *A* and *B* within quadrant pairs 0 and 2, 1 and 2, 300 and 302, and 301 and 302 would be passed to the next phase of the algorithm, in sequence, for further processing.

(2) Locate closest point pair. For each pair of facing quadrants located by phase (1), do the following: first, subdivide each of these two quadrants. Then, compare each of the four resulting descendant quadrants of one quadrant with all descendant quadrants in the adjacent quadrant, keeping all candidate "nearest" pair(s) that contain portions of *A* in one and *B* in the other according to a pairwise quadrant nearness hierarchy (this will be described in detail next). This has the effect of discarding other pairwise combinations that cannot possibly contain portions of a feature that are nearer than those kept. If a selected pair is not already at the finest subdivision level, then recursively repeat phase (2) for each of the resultant "nearest" candidate pairs until arriving at the finest level of subdivision, where a point-to-point distance calculation can be made between candidate lowest-level pairs. Otherwise, the pair is already at the finest level of subdivision. In this situation, calculate the Euclidean distance between the two quadrant locations. Then, select minimum Euclidean distance between the distance measure just calculated and the minimum previously calculated to this point.

The minimum distance is initialized to infinity at the start of the entire process. As in phase (1), quadrant pairs being examined at any given time always will be at the same level of the hierarchy.

The quick-estimate distance comparison via a relative nearness hierarchy

Use of relative nearness for selection of candidate "nearest" quadrant feature location pairs within an adjacent pair of quadrants can best be seen graphically. Figures 5 and 6 show the potential

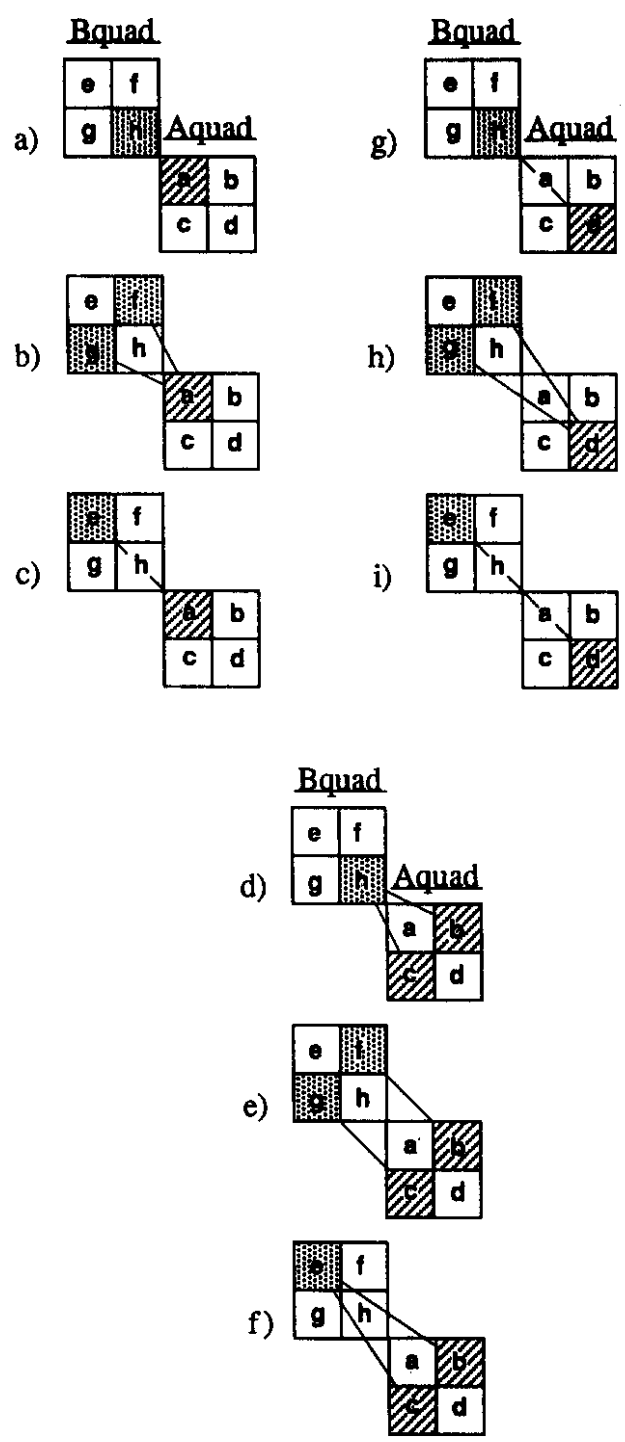


Figure 6. Nearness hierarchy used in selecting "nearest" candidate subquadrant pairs within diagonally adjacent quadrants.

configurations of candidate "nearest" pair sets for subquadrants of adjacent quadrant pairs. Cardinal and diagonal neighbors represent two distinct but logically similar situations. Figure 5 shows the configurations for East–West and Figure 6 shows the configurations for the relative nearness quick-estimate selection for quadrants in a Southeast–Northwest relative orientation.

The quadrant containing a portion of feature *A* is termed Aquad and the other quadrant of the pair, containing some portion of feature *B*, is termed Bquad. To locate the nearest subquadrant cell pairs between *A* and *B* quadrants, the basic logic is to check closest neighboring subquadrants first and if one or both of the subquadrants in Aquad and Bquad do not contain some portion of their respective feature, to progress sequentially to increasingly distant pair combinations of subquadrants. For the sake of clarity in the following description of how this process operates, the subquadrants are labeled with unique letter identifiers in place of their numerical quadtree addresses (cf. Figs. 5 and 6).

For cardinal neighbors where some portion of *a* and/or *c* (i.e. facing side of Aquad) is occupied, the process proceeds as follows (cf. potential configurations in Figs. 5A–5C): if cell *a* is not empty (i.e. contains a portion of feature *A at any quadtree level* within it), then cell *f* is tested first for the presence of a portion of feature *B*. If so, Bquad[f] is selected as the closest neighboring subquadrant to Aquad[a] regardless of other cells that may be occupied, and the process skips to locate the nearer nonempty cell(s) relative to Aquad[c]. (Note that if cells *a* and *f* do contain some portion of their respective features, then *c–h* is the only additional potential nearest cell pair that can be selected.) If cell *f* is empty, however, and cell *b* is also empty, then cells *e* and *h* are examined. If either *e* or *h* are occupied, the cell pairs of *a–e* or *a–h* are selected, accordingly. If both *e* and *h* are occupied, then *both* are selected as possible nearest cells. (Both *a–e* and *a–h* are selected in this example to allow for the situation where we are not at the lowest level and the area of cell *h*, for example, is not completely occupied by feature *B*. If it is determined that only the southwestern corner of cell *h* is occupied after further subdivision, then any portion of feature *B* on the eastern edge of cell *e* would be at least as near to *a*.) If both *e* and *h* are empty and *b* also is empty, then the pair *a–g* is selected as indicated in Figure 5C.

The equivalent sequence then is performed also to locate the nearest nonempty cells in Bquad relative to Aquad[c], as shown in Figure 5D–5F.

Another alternative situation for this process as it relates to cardinal neighbors is the situation where cells *a* and *c* (i.e. the "near" side of Aquad) are both empty. Then, the same relative distance sequence holds for cells *b* and *d* (the "far" side of Aquad) relative to the subquadrants of Bquad, but for the configurations shown on the right side of Figure 5 (Figs. 5G–5L) instead.

The same basic logic of testing progressively more distant subquadrant pairs holds for the situation of diagonal neighbors, as shown in Figure 6. Also note that, although only the East–West and Southeast–Northwest relations between Aquad and Bquad are shown, this process is identical for all relative directional combinations.

Because of the top-down recursive nature of this process, Aquad and Bquad may not be physically adjacent after the top level of the recursion, when Aquad and Bquad are passed directly from phase (1). The relative nearness test for any Aquad–Bquad pair can result in as many as four candidate nearest subquadrant pairs. This would happen, for example, if *a*, *c*, *f*, and *h* were all occupied because it is not known where a given feature lies within a given subquadrant unless we are at the pixel level. In practice, most of these potential nearest pairs are eliminated when the cells are subdivided further and the process is repeated.

Table 1. Example based on polygons shown in Figure 4

First subdivision		
Quadrant	A	B
0	031	[empty]
1	103	[empty]
	120	
	121	
	1223	
	123	
	130	
2	[empty]	212
		213
		230
		2310
		2311
3	3001	3020
	3010	3022
		3200
Second subdivision		
Quadrant	A	B
0	3001	3020
	3010	3022
1	[empty]	[empty]
2	[empty]	3200
3	[empty]	[empty]
Third subdivision		
Quadrant	A	B
0	3001	[empty]
1	3010	[empty]
2	[empty]	3020
		3022
3	[empty]	[empty]

Implementation utilizing leafcodes to represent location sets

It is assumed that the algorithm is implemented using input that consists of two lists of maximal block encoded leafcode addresses, as shown in Figure 4. Each list (i.e. before any subdivisions are performed) represents the complete locational extent of one of the features. These lists always are maintained in depth-first order. Note that the two features need not be resident in the same map or image data file, but may be stored in two spatially registered images where the leafcode addresses act as the common coordinate system. By using address lists representing the two features, much unoccupied space simply can be ignored.

(1) Locate adjacent quadrant pairs. The subdivision step involves separating the list of addresses for each of the two features into four lists, one for each subquadrant of the quadrant containing both features. For each of the input lists, allocate each address within it to one of the four lists on the basis of the leftmost digit that had not been examined previously, as shown in the example of Table 1 based on the polygons shown in Figure 4. Note that because feature *B* is completely within subquadrants 2 and 3, the "*B*" lists for subquadrants 1 and 2 are empty. Similarly, because feature *A* extends into all but subquadrant 2, only the "*A*" list for subquadrant 2 is empty.

The quadrant comparison step is accomplished by comparing all combinations of the address list for *A* relative to each quadrant with the address list of *B* relative to each quadrant (i.e. A0 vs B0, B1, B2, and B3, etc.), in sequence. If both *A* and *B* lists for the same subquadrant (i.e. A0 vs B0, A1 vs B1, etc.) are not empty, then subdivide again and repeat the process with the resulting lists. If one or both of any pair combination is empty, discard both lists. Otherwise, we have separate portions of the polygon in adjacent subquadrants that can be processed further to determine the closest pair of cells. Note that this may include a quadrant pair where either quadrant may *also* contain both features. This occurs in Figure 4 where separate parts of features *A* and *B* are in quadrants 0 and 3, respectively, but quadrant 3 also contains a portion of feature *A*. In this situation, quadrants 0 and 3 will be passed directly to the next step to locate the closest pair of points relative to that pair of quadrants. When quadrant 3 relative to *A* and quadrant 3 relative to *B* are compared, this quadrant then will be subdivided and candidate nearest pairs will be determined for appropriate subquadrants.

(2) Locate closest point pair. Given leafcode lists for the contents of a pair of adjacent quadrants, each list is subdivided into four subquadrant lists using the same process as in phase (1). If either input list for an adjacent quadrant contains only the address of that entire quadrant (i.e. is completely occupied by one of the features), that quadrant is subdivided also. The appropriate subquadrant addresses are generated by replicating the address of the parent quadrant four times and appending a subquadrant digit to each. This is done so that the lowest level facing edge cells are located and interior space quickly eliminated, regardless of the size of the original input cells. The closest subquadrants in one quadrant relative to any subquadrant in the other can be determined easily utilizing the relative nearness sequence as described. For the sake of brevity and clarity utilizing leafcode lists, it is best to express this logic relative to a limited set of relative orientations, as just described, and perform a coordinate rotation prior to performing this relative nearness comparison step at the next subdivision level.

In the present context, rotation of leafcodes about the center of a common parent quadrant for the purpose of normalizing the inputs is a simple procedure. The relative subquadrant locations of Aquad and Bquad are given by the nth digit from the left, where n = quadtree level, or depth, of the current subdivision. The digit always will be the same number for all addresses within Aquad, and similarly for Bquad. In fact, this digit and all digits to the left will be the same for all leafcodes within Aquad and within Bquad. For example, if the leafcode list for Aquad is {301, 303} and for Bquad is {330, 332, 333} with depth = 2, then Aquad and Bquad are NW and SE

of each other, respectively, because the second digit from the left in address of Aquad is 0 and in Bquad is 3. Once the relative position is determined in this manner, the appropriate address rotation can be made to the normalized relative directions in a simple, one-to-one mapping.

Because the distance algorithm operates on relative subquadrant positions for one level at a time, there is no need to perform this rotation on the actual leafcode addresses. Instead, all leafcodes contained in each of the four subquadrants for both Aquad and Bquad are allocated to the appropriate list of unique letter-identified quadrant list. For the purpose of a formal description of the algorithm, rotation to standardized E–W cardinal directions and SE–NW diagonal directions are used, as previously discussed.

The algorithm

In the formal definition of the distance algorithm that follows, it is assumed that the address lists for A and B relate to a common, level 0 parent quadrant that completely contains the two polygons. In other words, the outer edge of the common parent quadrant can be viewed as a bounding square around the two features.

The procedures **DISTANCE** and **PAIRDIST** correspond to the logic of phases (1) and (2), respectively. **DISTANCE** returns the minimum point-to-point Euclidean distance as its value and the addresses of the two closest cells between the polygons as **Apt** and **Bpt**. **SUBDIVIDE** takes a single list of leafcodes as input and separates it into four separate lists that contain the addresses within each of the four subquadrants at the given **depth** (i.e. address digit). If an address is input that is at the next higher level (i.e. fewer digits) than the current value of **depth**, then the appropriate subquadrant addresses are generated. **GET_QUADLOC** inputs a single leafcode address and returns the single digit in the position indicated by **depth**. **DISTNC** computes the point-to-point Euclidean distance between the two input leafcode addresses. This routine also performs the leafcode to Cartesian x–y conversion necessary in the linear quadtree context prior to the distance calculation itself utilizing the standard equation for point-to-point Euclidean distance.

In the initial call to the top-level recursive procedure, **depth** = **1**. **Maxdepth** represents the maximum number of subdivisions (and thus also the maximum number of address digits) allowed in the current quadtree hierarchy. The value of the constant **diagonal** used in **PAIRDIST** is 3. This is used as a test value to identify a diagonal pair, since the digits in the two leafcodes corresponding to the current level being examined always will sum to 3 only if they are on a diagonal in relation to each other (cf. Fig. 3).

```
procedure DISTANCE (A, B, nearestApt, nearestBpt, depth)

  /*  Given two arbitrarily-shaped features, return the minimum euclidean distance between the two features.
      It is assumed that A and B are passed to this function and each contains a list of maximal-block encoded
      quadtree addresses that define the complete areal extent of each feature, respectively. It is also assumed
      that both lists can be subdivided at least once (i.e., depth < maxdepth). */
begin

    Mindist ← INFINITY;

    /* subdivide quadtree coordinate lists A & B each into 4 subquadrant lists; asub[0] through asub[3] and
                 bsub[0] through bsub[3]. */
    subdivide(A, asub[0], asub[1], asub[2], asub[3], depth);
    subdivide(B, bsub[0], bsub[1], bsub[2], bsub[3], depth);
    depth ← depth + 1

    for i ← 0 to 3 do
        for j ← 0 to 3 do /* for all pair combinations of subquadrants at the current level */
            begin
                if asub[i] ≠ EMPTY  and  bsub[j] ≠ EMPTY then
                    if i = j then  /* recurse */
                        Dist ← distance (asub[i], bsub[j], Apt, Bpt, depth +1);
                    else  /* i ≠ j - find closest point pair & their dist.*/
                        Dist ← pairdist (asub[i], bsub[j], Apt, Bpt, depth);

                    if Dist < Mindist then /* find minimum distance so-far &
                                 record corresponding addresses */

                        begin
                            Mindist ← Dist;
                            nearestApt ← Apt;
                            nearestBpt ← Bpt;
                        end;
            end;
  return Mindist;
end;
```

```
procedure PAIRDIST(Aquad, Bquad, Apt, Bpt, depth)

/* recursively find the closest pair of occupied, lowest-level cells between two given quadrant locations of Aquad
& Bquad. The return value of the function is the minimum euclidean distance between the pair of closest cells.
The addresses of the nearest pair of cells are returned as the first addresses in Aquad & Bquad. */

begin

  mindist ← dist ← INFINITY;
  Bnear[0] ← Bnear[1] ← Bnear[2] ← Bnear[3] ← EMPTY;

  if depth < MAXDEPTH then /* we're not at the lowest level yet */
      if Aquad ≠ EMPTY  and  Bquad ≠ EMPTY then
          begin
              Aquadloc ← get_quadloc(Aquad, depth)
              Bquadloc ← get_quadloc(Bquad, depth)
 subdivide (Aquad,asub[0],asub[1],asub[2],asub[3], depth);
 subdivide (Bquad,bsub[0],bsub[1],bsub[2],bsub[3], depth);
 depth ← depth + 1
 /* if we don't have a diagonal pair, then perform the appropriate rotation to a standardized
 cardinal orientation, inserting dummy alphabetic coordinate list names.  B west of A is used as
 the standardized directional relationship for cardinal neighbors. */
 if (Aquadloc + Bquadloc) ≠ DIAGONAL then
     begin
         /* B west of A ? */
         if (Aquadloc = 1 and Bquadloc = 0) or
                (Aquadloc = 3 and Bquadloc = 2) then
             begin
                 a ← asub[0]; b ← asub[1]; c ← asub[2]; d ← asub[3];
                 e ← bsub[0]; f ← bsub[1]; g ← bsub[2]; h ← bsub[3];
             end;
         /* B east of A ? */
         else if (Aquadloc = 0 and Bquadloc = 1) or
           (Aquadloc = 2 and Bquadloc = 3)  then
             begin
                 /* copy subquadrant address lists into their appropriate places */
                 a ← asub[1]; b ← asub[0]; c ← asub[3]; d ← asub[2];
                 e ← bsub[1]; f ← bsub[0]; g ← bsub[3]; h ← bsub[2];
             end;

         /* B south of A ? */
         else if (Aquadloc = 0 and Bquadloc = 2) or
                (Aquadloc = 1 and Bquadloc = 3) then
             begin
                 a ← asub[2]; b ← asub[0]; c ← asub[3]; d ← asub[1];
                 e ← bsub[2]; f ← bsub[0]; g ← bsub[3]; h ← bsub[1];
             end;
         /* else, we must have B north of A. */
         else  /* we know that (Aquadloc = 2 and Bquadloc = 0) or
                 (Aquadloc = 3 and Bquadloc = 1) */
             begin
                 a ← asub[1]; b ← asub[3]; c ← asub[0]; d ← asub[2];
                 e ← bsub[1]; f ← bsub[3]; g ← bsub[0]; h ← bsub[2];
             end;


     /* set up Aquad side for potential "nearest" matches */
     if a ≠ EMPTY  or  c ≠ EMPTY then
         begin /* "nearest" candidate pair(s) must match with a & c if at least one is not
         empty */
             Anear[0] ← a; Anear[1] ← a;
             Anear[2] ← c; Anear[3] ← c;
         end;
     else
       begin /* otherwise, the "nearest" candidate pair(s) must match with b & d */
           Anear[0] ← b; Anear[1] ← b;
           Anear[2] ← d; Anear[3] ← d;
       end;

 /* match up Bquad side for individual "nearest" candidate pairs*/
 if Anear[0] ≠ EMPTY then
     if f ≠ EMPTY then Bnear[0] ← f;
```

```
        else if Anear[2] = EMPTY then
            begin
                if e ≠ EMPTY  or  h ≠ EMPTY then
                    begin
                        if e ≠ EMPTY then Bnear[0] ← e;
                        if h ≠ EMPTY then Bnear[1] ← h;
                    end;
                /* this last test is to anticipate a closer possible h-c match */
                else if h = EMPTY then Bnear[0] ← g;
            end;
        else if h = EMPTY then Bnear[0] ← e;
    if Anear[2] ≠ EMPTY then
        if h ≠ EMPTY then Bnear[2] ← h;
        else if Anear[0] = EMPTY then
            begin
                if f ≠ EMPTY  or  g ≠ EMPTY then
                    begin
                        if g ≠ EMPTY then Bnear[2] ← g;
                        if f ≠ EMPTY then Bnear[3] ← f;
                    end;
                else if f = EMPTY then Bnear[2] ← e;
            end;
            else if f = EMPTY then Bnear[2] ← g;
        end; /* of procedure for cardinal directions */

else /* we have a diagonal pair */
/* perform appropriate rotation*/
    begin
        /* B northwest of A ? */
        if Bquadloc = 0 then
            begin
                a ← asub[0]; b ← asub[1]; c ← asub[2]; d ← asub[3];
                e ← bsub[0]; f ← bsub[1]; g ← bsub[2]; h ← bsub[3];
            end;
        /* northeast ? */
        else if Bquadloc = 1 then
            begin
                a ← asub[1]; b ← asub[3]; c ← asub[0]; d ← asub[2];
                e ← bsub[1]; f ← bsub[3]; g ← bsub[0]; h ← bsub[2];
            end;
/* southwest ? */
else if Bquadloc = 2 then
    begin
        a ← asub[2]; b ← asub[3]; c ← asub[0]; d ← asub[1];
        e ← bsub[2]; f ← bsub[3]; g ← bsub[0]; h ← bsub[1];
    end;
/* must be southeast */
else
    begin
        a ← asub[3]; b ← asub[2]; c ← asub[1]; d ← asub[0];
        e ← bsub[3]; f ← bsub[2]; g ← bsub[1]; h ← bsub[0];
    end;

/* given a standardized directional template, determine which cell pairs are to be
compared further, according to the cell nearness hierarchy */
if a ≠ EMPTY then
    begin
        Anear[0] ← Anear[1] ← a;
        if h ≠ EMPTY then Bnear[0] ← h;
        else if  g ≠ EMPTY  or  f ≠ EMPTY  then
            begin
                Bnear[0] ← g; Bnear[1] ← f;
            end;
        else Bnear[0] ← e;
    end;
else if c ≠ EMPTY  or  b ≠ EMPTY then
    begin
        Anear[0] ← c; Anear[1] ← b;
        Anear[2] ← c; Anear[3] ← b;
        if h ≠ EMPTY then
            begin
                Bnear[0] ← h; Bnear[1] ← h;
```

```
            end;
        else if g ≠ EMPTY or f ≠ EMPTY then

            begin
                Bnear[0] ← g; Bnear[1] ← g;
                Bnear[2] ← f; Bnear[3] ← f;
            end;
        else
            begin
                Bnear[0] ← e; Bnear[1] ← e;
            end;
    end;
else
    begin
        Anear[0] ← d; Anear[1] ← d;
        if h[0] ≠ EMPTY then Bnear[0] = h;
            else if g ≠ EMPTY or f ≠ EMPTY then
                begin
                    Bnear[0] ← g; Bnear[1] ← f;
                end;
            else Bnear[0] ← e;
                end;
            end;

        /* now, calculate pairwise distances for all candidate pairs and find the minimum distance */
        for i ← 0 to 3 do
            begin
                dist ← pairdist(Anear[i],Bnear[i], Apt, Bpt, depth);
                /* recurse to get a lowest-level pair */
                if dist < mindist then
                    begin
                        mindist ← dist;
                        nearestApt ← Apt;
                        nearestBpt ← Bpt;
                    end;
            end;
    end; /* of procedure for non-null input */
    return mindist;

  else /* we came in at the lowest level */
    begin
        nearestApt ← Aquad;
        nearestBpt ← Bquad;
        return distnc(Aquad, Bquad);
    end;
end;
```

DISCUSSION AND EVALUATION

The algorithm described here can be used to determine the minimum Euclidean distance between two features of arbitrary shape when these features are represented in quadtree form. It functions correctly if either or both features are polygons, lines, points, or sets of points. Polygons can have holes and lines can have breaks. This directly follows from the view of the basic problem as locating the closest pairs of points between two point sets where the point sets represent a complete locational representation of the two features.

The algorithm is a top-down recursive procedure that always examines the contents of paired quadrants at the same relative subdivision level. The actual Euclidean distance measure is calculated only on the basis of lowest-level cells. If **PAIRDIST** locates a "nearest" candidate pair of cells that are not at the lowest level, **PAIRDIST** is called recursively to subdivide further and eliminate cells until locating a "nearest" candidate pair of cells at the lowest level. The recursion then unwinds until all lowest level pairs are located for the two facing quadrants originally passed from **DISTANCE**.

Analyzing the quadrant distance algorithm from the inside-out, the theoretical worst-case execution time can be calculated as follows: within **PAIRDIST**, the amount of time required for rotation into standardized relative directions and identification of the specific "nearest" subquadrant pair within a given pair of quadrants is constant. The amount of time

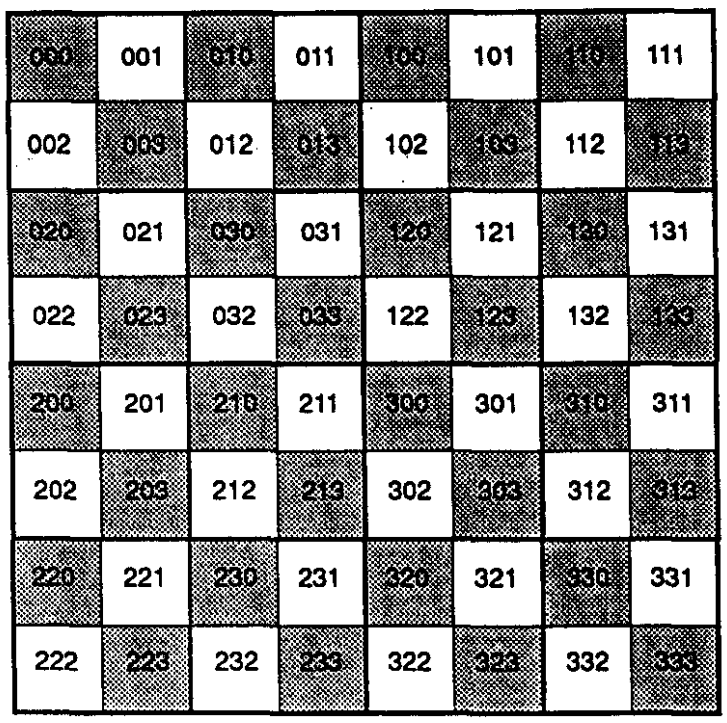


Figure 7. Worst case configuration: shaded blocks represent locations of one feature and the nonshaded blocks locations of other.

required for the actual distance calculation and subsequent comparison within **DISTANCE** of this measurement to the minimum located so-far also is constant. Each of these four steps occurs in sequence, and combined represent an $O(K)$ process overall where K is some constant. Every candidate pair of quadrants must be subdivided to the lowest level "nearest" subquadrant pair, eliminating other pairs at each level through the subdivision process, before the actual distance calculation and comparison can be performed. This can be viewed conceptually as a preorder, top-down search of the quadtree to locate the two closest points, which requires $O(\log n)$ steps, where $n =$ the total number of nodes in the tree, including the lowest level nodes. If *all* nonpixel level nodes must be subdivided to locate the candidate "nearest" lowest level subquadrant pairs (i.e. no subquadrants can be eliminated in either phase of the algorithm), this would represent the worst case and would require $O(n - p(\log n))$ steps, where $p =$ the number of pixel-level nodes.

The spatial configuration of the worst-case as mentioned would be a perfectly regular, pixel-level checkerboard distribution of two intertwined features, as shown in Figure 7. This can be described as a perfectly interspersed point set. This case, however, does not require the theoretically maximum number of steps because some diagonal pairs are discarded as candidates if a cardinal neighbor also is present. A more important observation from this analysis is that overall performance for the algorithm is most affected by the number of facing node pairs that must be subdivided. Thus the speed of the algorithm is affected primarily by the length of the facing sides of the two features. This, of course, is a function of the relative position of the two features as much as their total areal extent. The more closely the two features are aligned, the more steps (and time) will be required for determining minimum distance. The number of steps also is affected by the number of levels in the quadtree that must be traversed for each facing node pair, but to a decreasing degree as the number of levels increases. It also is noted that the time required to calculate distance by this algorithm is not affected by the relative distance of the two features, unlike region expansion.

The top-down recursive nature of this algorithm and the data model on which it is based can be used to shorten the time needed to calculate minimum distance by simply not descending to the pixel level, but rather stopping at some specified maximum-depth that is above the pixel level. This is why the term **maxdepth**, or "lowest level", was used in the above algorithm description rather than **pixel**. It is noted, however, that this introduces a directional bias in addition to an increased degree in imprecision because of the need to use the cell centerpoint location for calculating Euclidean distance. This is not a problem when descending completely down to the pixel level with the assumption that the pixel resolution is suitably small. Thus, not descending to the pixel level provides a means for obtaining quick distance estimates.

SUMMARY

The algorithm presented in this paper offers an efficient alternative solution for finding minimum distance between *any* two features of arbitrary shape and configuration. A primary advantage of this algorithm, perhaps more than efficiency, is its ability to provide rapid approximate distance measures as differing levels of resolution.

REFERENCES

Ang, C. H., and Samet, H., 1988, Fast region expansion for quadtrees: Proc. Third Intern. Geographical Union Symposium on Spatial Data Handling, Sydney, Australia, p. 19–37.

Ang, C. H., Samet, H., and Shaffer, C. A., 1990, A new region expansion for quadtrees: IEEE Trans. Pattern Analysis and Machine Intelligence, v. 12, no. 7, p. 682–686.

Bentley, J., 1980, Multidimensional divide-and-conquer: Commun. Assoc. Comput. Mach., v. 23, no. 4, p. 214–229.

Bobrow, J. E., 1989, A direct minimization approach for obtaining the distance between convex polyhedra: Intern. Jour. Robotics Research, v. 8, no. 3, p. 65–76.

Chin, F., and Wang, C., 1984, Minimum vertex distance between separable convex polygons: Info. Processing Letters, v. 18, p. 41–45.

Forsgren, P. O., and Seideman, P., 1990, An interobject distance measure based on medial axes retrieved from discrete distance maps: IEEE Trans. Pattern Analyses and Machine Intelligence, v. 12, no. 4, p. 390–397.

Fortune, S., 1986, A sweepline algorithm for Voronoi diagrams: Proc., Second ACM Symposium on Computational Geometry, Yorktown Heights, New York, p. 313–319.

Gargantini, I., 1982, An effective way to represent quadtrees: Commun. Assoc. Comput. Mach., v. 25, no. 12, p. 905–910.

Guibas, L., and Stofoli, J., 1985, Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams: ACM Trans. Graphics, v. 4, p. 74–123.

Klinger, A., 1971, Patterns and search statistics, *in* Rustagi, J. S., ed., Optimizing methods in statistics: Academic Press, New York, 488 p.

van Lierop, M., 1986, Geometrical transformations of pictures represented by leafcodes: Comput. Vision, Graphics, Image Processing, v. 33, p. 81–98.

Morton, G., 1966, A computer oriented geodetic data base and a new technique in file sequencing: unpublished manuscript, IBM, Ltd, Ottawa, Canada, 20 p.

Peuquet, D., 1984, A conceptual framework and comparison of spatial data models: Cartographica, v. 21, no. 4, p. 66–113.

Peuquet, D., 1988, Representations of geographic space: toward a conceptual synthesis: Annals Assoc. Am. Geographers, v. 78, no. 3, p. 375–394.

Preparata, F. P., and Shamos, M., 1985, Computational geometry: an introduction: Springer-Verlag, New York, 459 p.

Rosenfeld, A., and Pfaltz, J. L., 1968, Distance functions on digital pictures: Pattern Recognition, v. 1, p. 33–61.

Samet, H., 1982, Distance transform for images represented by quadtrees: IEEE Trans. Pattern Analysis and Machine Intelligence, v. PAMI-4, no. 3, p. 298–303.

Samet, H., 1989a, Applications of spatial data structures: Addison-Wesley Publ. Co., Reading, Massachusetts, 507 p.

Samet, H., 1989b, The design and analysis of spatial data structures: Addison-Wesley Publ. Co., Reading, Massachusetts, 493 p.

Schwartz, J., 1981, Finding the minimum distance between two convex polygons: Info. Processing Letters, v. 13, p. 168–170.

Shamos, M., 1978, Computational geometry, unpubl. doctoral dissertation, Dept. Computer Sci., Yale Univ., 231 p.

Tomlin, C. D., 1990, Geographic information systems and cartographic modeling: Prentice-Hall, Englewood Cliffs, New Jersey, 249 p.

Wang, C. A., and Chan, E., 1986, Finding the minimum visible vertex distance between two nonintersecting simple polygons: Proc. Second Annual Symposium on Computational Geometry, Yorktown Heights, New York, p. 34–42.

Widmayer, P., Wu, Y. F., and Wong, C. K., 1985, Distance problems in computational geometry with fixed orientations: Proc. Symposium on Computational Geometry, Baltimore, Maryland, p. 186–195.